NetworkAnalysis-master.zip ZIP

Melanoma Genes.xlsx XLSX

CRC_Genes.xlsx XLSX

I have this whole folder with me given by my project lead and he has assigned me the following user story to work on -
Test the existing network proximity script with the pair of drugs below:
Dabrafenib (BRAF) + Trametinib (MEK) in melanoma
Cetuximab (EGFR) + Vemurafenib (BRAF) in colorectal cancer (CRC)
generate both excel and power point version of the data.

I think inside this folder i have to focus only on Network_proximity folder for this user story, I am not sure as these scripts are written by someone else so.....
also i think the scripts that i have generates only excel version & not the power point version - I will give that script for power point version later on (confirm this also pls)

Opening the uploaded files to see what they contain.
I'll look through the folder first to see what's in it and confirm both of your questions.

Ran 5 commands, read a file ›
I've been reading through the whole folder and checking it against your two gene files, and I've now seen enough to answer you.

Which folder to focus on: You're right, only `Network_proximity` matters. `drug_synergy.py` is the main script. `precompute_shortest_paths.py` builds the network cache it needs. `kegg_id.py` is a side utility you won't need. The other folders (centrality, hub, shortest_path, WGCNA, Topology_analysis, vulnerability_analysis) are separate analyses. `use-case/` is a worked example from an earlier run, and it shows the exact input format to copy.

Your Excel/PowerPoint question: Not quite. The script produces neither. It writes a **CSV** (`Drug_combination_results.csv`) and a log file. Excel is only used as an *input* (`Drug_target.xlsx`). So the Excel output also has to be generated, which is easy to do from the CSV. You're right that the PowerPoint script will come separately.

What the script expects as input: a `Drug_target.xlsx` with two sheets.

- `Module_genes` : one column per disease, with the disease's genes listed below. Your Melanoma (485 genes) and CRC (559 genes) files fit this once reshaped.
- `Drug_combination` : columns `Disease` , `Drug_target1` , `Drug_target2` , where the disease name must match the column header exactly. It takes gene symbols, not drug names:

- Melanoma: BRAF + MAP2K1,MAP2K2 (Dabrafenib + Trametinib)
- CRC: EGFR + BRAF (Cetuximab + Vemurafenib)

Trametinib's MEK target maps to two genes, MAP2K1 and MAP2K2, and both are in your melanoma list. Please confirm with your lead that this mapping is what he wants.

The blocker: the script can't run yet, because two files it loads are not in the zip.

1. `STRING_PPI_filtered.txt`: the filtered STRING network, tab-separated with `source`, `target` and a `Weight` column.
2. `shortest_paths_cache.pkl`: generated from that network by `precompute_shortest_paths.py`. The example log shows it covering about 12,000 nodes, and the precompute step is slow.

My sandbox has no internet access, so I can't download STRING myself. Please ask your lead for `STRING_PPI_filtered.txt`, and for the `.pkl` too if he already has it. That would save you the precompute run.

Things to ask or watch:

- The script treats a shortest-path distance of 0.0 as a "direct neighbour", and the precompute step defaults edge weights to 0.0. I'd check with your lead that the weights in the filtered STRING file are meant to be used this way.
- No random seed is passed, so the z-scores and p-values will shift slightly between runs. That's expected.
- Some genes in your lists (e.g. `MT-CO1`) may not be in STRING. The script will log a warning and drop them.

Once you send the STRING file (or the `.pkl`), I can build the input workbook, run both combinations, and write the Excel output. I can also lay the results out in a clean structure so your PowerPoint script can pick them up later.

STRING_PPI_filtered.txt
will it be disease specific or what like if i were to download manually from internet
pls guide me step by step
what is this file about?

also please list down inputs, outputs, workflow - i am getting confused
also pls explain me each script line by line that i will be using for my user story please

Deciding whether to search for current download steps.
Let me quickly check the current STRING download page so the steps I give you are accurate.

Searched the web ›

Here's the plan: first what the STRING file is and how to build it, then the inputs/outputs/workflow overview, then the two scripts block by block. I did a quick check of the STRING download page, so the file names below are current (v12.0).

---

1. What is `STRING_PPI_filtered.txt`?

STRING is a public database of protein–protein interactions (PPI): which proteins physically or functionally interact, with a confidence score (`combined_score`, 0–1000) for each pair. The script uses it as a **map of the human cell**. Everything it computes is a distance between proteins on that map.

It is not disease-specific. It's one human network, reused for every disease and drug. The disease and drug come in separately: the disease genes (your Melanoma/CRC lists) and the drug targets are *placed onto* this network, and the script measures how close they are.

"Filtered" means the author dropped low-confidence edges. The example log shows 12,268 nodes, well under the full human STRING (roughly 19–20k proteins), so a cutoff was clearly applied. I don't know which one, so **ask your lead the cutoff (e.g. 400/700/900) and how the `Weight` column was calculated.** That affects your results.

Required format: tab-separated, header `source`, `target`, `Weight`, with **gene symbols** (BRAF, EGFR), not STRING protein IDs.

Building it manually

1. Open string-db.org → **Download**, choose organism *Homo sapiens (9606)*, version 12.0 (match your lead's version if it differs).
2. Download two files:
   - `9606.protein.links.v12.0.txt.gz` (~79 MB), with columns `protein1 protein2 combined_score`
   - `9606.protein.info.v12.0.txt.gz` (~2 MB), which maps protein IDs to gene symbols (`preferred_name`)
3. Convert them with a short script:

python

```
import pandas as pd
links = pd.read_csv("9606.protein.links.v12.0.txt.gz", sep=" ")
info  = pd.read_csv("9606.protein.info.v12.0.txt.gz", sep="\t")
id2name = dict(zip(info["#string_protein_id"], info["preferred_name"]))

links = links[links["combined_score"] >= 700]        # <- cutoff: confirm with
links["source"] = links["protein1"].map(id2name)
links["target"] = links["protein2"].map(id2name)
links = links.dropna(subset=["source", "target"])
links = links[links["source"] < links["target"]]     # drop A-B / B-A duplicate
links["Weight"] = ...                                # <- confirm with lead
links[["source","target","Weight"]].to_csv("STRING_PPI_filtered.txt", sep="\t",
```

4. Run `precompute_shortest_paths.py` on it to create `shortest_paths_cache.pkl`.

---

2. Inputs, outputs, workflow

Stage	File	Role
Input	`STRING_PPI_filtered.txt`	The human PPI network (same for all diseases)
Input	`Drug_target.xlsx`	Sheet `Module_genes` (one column per disease) and sheet `Drug_combination` (Disease, Drug_target1, Drug_target2)
Intermediate	`shortest_paths_cache.pkl`	Pre-computed distance between every pair of proteins
Output	`Drug_combination_results.csv`	Z-scores, separation score, p-value, classification per pair
Output	`drug_synergy_analysis.log`	Run log
To create	Excel and PowerPoint versions	Not made by the scripts (Excel from the CSV; PPT later)

Workflow

1. Get or build `STRING_PPI_filtered.txt`.
2. Run `precompute_shortest_paths.py` → `shortest_paths_cache.pkl` (once; reused for both diseases).
3. Build `Drug_target.xlsx`: put the Melanoma and CRC gene lists in `Module_genes` (columns named `Melanoma`, `CRC`), and add the pairs to `Drug_combination`:
   - Melanoma: `BRAF` / `MAP2K1,MAP2K2`
   - CRC: `EGFR` / `BRAF`

   Write multiple genes **without spaces** after the comma (see the first flag below).
4. Run `drug_synergy.py` → CSV + log.
5. Convert the CSV to Excel, then PPT with the script you'll receive.

---

3. Line-by-line walkthrough

`precompute_shortest_paths.py` (precompute step)

- **Imports:** `networkx` builds the graph, `pandas` reads the file, `pickle` saves the result, `tqdm` shows a progress bar. You'll need `pip install tqdm networkx pandas openpyxl numpy`.
- `df = pd.read_csv(..., sep='\t')`: reads the edge list.
- `network = nx.Graph()`: an undirected graph.
- **Loop over rows:** converts source/target to strings and reads `Weight` (if the column exists and isn't blank; otherwise uses `weight_default`, 0.0). `add_edge` adds the

edge.

- **Print node/edge counts:** a sanity check.
- `single_source_dijkstra_path_length`: for each protein, computes the shortest weighted distance to every reachable protein, stored as `{protein: {other: distance}}`.
- `pickle.dump`: saves it. This is slow and memory-heavy (~12k × 12k pairs).
- `__main__`: file names are hard-coded, so run it in the folder containing the STRING file.

`drug_synergy.py` (main script)

Setup (top): imports and logging that writes to both the screen and `drug_synergy_analysis.log`.

`NetworkProximityAnalyzer` class

- `load_shortest_paths`: loads the pickle into `self.shortest_paths`.
- `get_network_nodes`: the set of all proteins in the network.
- `get_degree_binning`: counts each node's "degree" as the number of entries with distance `== 0.0`, groups nodes by degree, and merges neighbouring degrees until each bin has at least 100 nodes. This is used so random nodes resemble real ones in connectivity.
- `get_degree_equivalents`: for each of your nodes, lists the other nodes in its degree bin.
- `get_random_nodes_matching_degree`: builds 1000 random sets, swapping each real node for a random same-bin node (falls back to any node).
- `calculate_closest_distance`: for each node in set A, finds the nearest node in set B, then averages those minima. This is the "proximity" distance.
- `calculate_proximity`: keeps only genes that exist in the network, computes the real distance, repeats it for 1000 random drug/disease set pairs, then z = (real − random mean) / random std. **Negative z means closer than chance.**
- `calculate_separation` (with `calculate_within_set_distance` and `calculate_between_set_distance`): $s_{AB} = d_{AB} - (d_{AA} + d_{BB})/2$, where d_{AA} and d_{BB} are the average distances within each drug's target set and d_{AB} the average between the sets. **Negative means the drugs' targets overlap; positive means they're separate.**
- `calculate_empirical_p_value` / `calculate_separation_p_value`: the fraction of random runs at least as extreme as the real value.
- `classify_drug_combination_with_confidence`: runs everything for drug A and drug B and applies three yes/no tests: A near disease ($z < 0$), B near disease ($z < 0$), and drugs overlap ($s_{AB} < 0$). The combination of those gives the label (Overlapping, Complementary, Indirect, Single, Independent Action, Non-exposure), plus "Significant" or "but not significant" based on the separation p-value (< 0.05).

Helpers and `main()`

- `validate_moduleprotein` / `validate_targetprotein`: warn about genes missing from STRING.

- `main()`:
  1. loads the pickle and STRING.
  2. reads both sheets of `Drug_target.xlsx`.
  3. loops over each disease column and picks that disease's combinations.
  4. splits target strings on commas.
  5. expands each target gene to **itself plus all its direct STRING neighbours**.
  6. runs the classifier and appends the result.
  7. saves the CSV, with `ERROR` rows on failure.

Things I noticed that you should raise with your lead

1. **Spaces in targets:** `"MAP2K1, MAP2K2"` would fail, because the script only strips the whole string. Write `MAP2K1,MAP2K2`.
2. **Degree calculation looks off (by my reading):** "direct neighbour" is tested as distance `== 0.0`. If edge weights are non-zero, that only matches the node itself, so every node gets degree 1, there's one big bin, and the randomisation isn't degree-matched. If weights are all 0, every distance is 0. Either way it's worth confirming how the author's file was built.
3. **Drug-disease p-values don't change the label:** in the classifier, the "drug A and B significant" and "not significant" branches assign identical labels, so only the separation p-value matters. The log text differs, the result doesn't.
4. **No random seed:** results shift slightly between runs, and each combination takes about a minute.

I skipped `kegg_id.py` since it's not part of your story. If you'd like, I can turn this walkthrough into an annotated copy of both scripts with comments, saved as a file. Once you have the STRING file, send it over and I'll build the input workbook and run both pairs.

Filter interactions based on the combined confidence score:
While all links in STRING are meaningful, some analyses might require only higher-confidence interactions, excluding the more exploratory part of the interaction spectrum. This can reduce file sizes by up to 90%. Note: Confidence scores in the files below are multiplied by 1000.
all interactions (0.150)medium confidence (0.400)high confidence (0.700)highest confidence (0.900)
Remove redundant interaction pairs (AB-BA):
By default, the links files encode interactions in both directions (AB and BA). However, since functional and physical interactions are symmetric, recording them once (AB) is sufficient without losing information. This reduces links file sizes by 50%. This does not apply to regulatory links, where column order signifies directionality (A->B).
includes AB and BA pairsincludes AB pairs only
Choose column delimiter:
Different analysis tools may require specific delimiters (column separators). Select your preferred delimiter type for the links files to ensure compatibility with your

analysis tools. Note: This choice does not impact the content or file size
i can see these options also - all interactions (0.150), medium confidence (0.400), high confidence (0.700), highest confidence (0.900)
in 2nd option remove redundnat - includes AB and BA pairs , other one is includes AB pairs only
3rd option - choose column delimiter - TXT, TSV, CSV
like in filter option -

Show more

Weighing a STRING cutoff against the missing-gene mapping ›

Those are the filters on STRING's download page. Here's what to pick for each, and one fix to my earlier code.

1. Confidence filter: pick "high confidence (0.700)", then confirm with your lead.
This is the usual standard for network analyses, and it keeps the file small. But the cutoff the author used isn't recorded in the folder, so I'm guessing here. The score cutoff directly changes which proteins and edges exist, so it changes your z-scores. Ask your lead which one he used, and if he doesn't remember, state "high confidence (0.700)" as your assumption in your notes. The page's scores are multiplied by 1000, so 0.700 means `combined_score >= 700`.

2. Redundant pairs: pick "includes AB pairs only".
The network is undirected, so A–B and B–A are the same edge, and the script builds an undirected graph anyway. You lose nothing, and the file is half the size.

3. Delimiter: pick TSV.
It makes no difference to the content. TSV is tab-separated, which is the format the script's own network file uses, and it's the safest to read. (The default TXT is space-separated.)

Also make sure you take the standard **full network links** file for *Homo sapiens (9606)*, not the physical-subnetwork or regulatory ones, unless your lead says otherwise. You also still need the separate `protein.info` file to map protein IDs to gene symbols.

Fix to my earlier code: I had a line `links = links[links["source"] < links["target"]]` to remove duplicates. With "AB pairs only" that line would wrongly delete about half your real edges. Use this version instead, which works whichever option you choose:

python

```
import pandas as pd
links = pd.read_csv("9606.protein.links.v12.0.txt", sep="\t")   # use your actu
info  = pd.read_csv("9606.protein.info.v12.0.txt.gz", sep="\t")
id2name = dict(zip(info["#string_protein_id"], info["preferred_name"]))

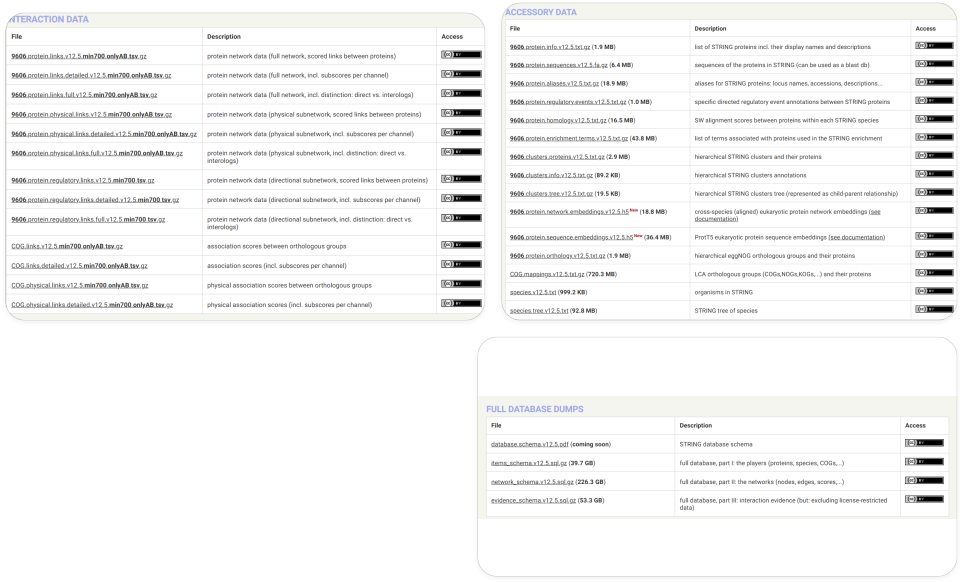
links = links[links["combined_score"] >= 700]        # harmless if already filt
links["source"] = links["protein1"].map(id2name)
links["target"] = links["protein2"].map(id2name)
links = links.dropna(subset=["source", "target"])

# remove A-B / B-A duplicates safely
pair = links[["source", "target"]].apply(lambda r: tuple(sorted(r)), axis=1)
links = links.loc[~pair.duplicated()]

links["Weight"] = ...                                # still to confirm with yo
links[["source","target","Weight"]].to_csv("STRING_PPI_filtered.txt", sep="\t",
```

If the downloaded links file has a different separator or name than I assumed, adjust the `read_csv` line. Check the first line of the file to see.

The `Weight` column is still open. STRING gives you a confidence score, but the script treats `Weight` as a *distance* (and uses 0.0 as a default), so how the author turned scores into weights matters. Please ask your lead that along with the cutoff. If you want, paste the first few lines of your downloaded file here and I'll check that the conversion works on it.



I got this after applying above - which to choose now

Choosing the file format and update code ›

Pick these two files, and ignore everything else on those pages.

1. From "Interaction data" (first screenshot): the first row
`9606.protein.links.v12.5.min700.onlyAB.tsv.gz`
This is the full human network with one score per protein pair. The name shows all three of your choices: `min700` (high confidence), `onlyAB` (no duplicate pairs) and `tsv` (tab-separated).

Why not the others:

- `detailed` adds a per-evidence-channel score breakdown that the script doesn't use.
- `full` separates direct evidence from interologs, which also isn't needed.
- `physical` is a smaller network of physical binding only. Your lead's analysis presumably used the full network, so confirm with him if unsure.
- `regulatory` is directional (A→B), which this script doesn't handle.
- `COG` files are for orthologous groups, not human proteins.

2. From "Accessory data" (second screenshot): the first row
`9606.protein.info.v12.5.txt.gz` (1.9 MB)
This maps STRING's protein IDs (like `9606.ENSP0000…`) to gene symbols such as BRAF. The links file only contains the IDs, so you need both files.

3. Third screenshot (full database dumps): don't download any of it. Those are 40–226 GB database dumps.

One note: the site now offers **v12.5**, while my earlier example said 12.0. That's fine, because it's just the newer release, but mention the version in your notes, since your lead's original file may have come from an older release and the numbers could differ slightly.

Since the filtering and duplicate removal are already done by the website, the conversion script gets simpler:

python

```
import pandas as pd

links = pd.read_csv("9606.protein.links.v12.5.min700.onlyAB.tsv.gz", sep="\t")
info  = pd.read_csv("9606.protein.info.v12.5.txt.gz", sep="\t")

id2name = dict(zip(info["#string_protein_id"], info["preferred_name"]))
links["source"] = links["protein1"].map(id2name)
links["target"] = links["protein2"].map(id2name)
links = links.dropna(subset=["source", "target"])

links["Weight"] = ...   # still to confirm with your lead

links[["source", "target", "Weight"]].to_csv("STRING_PPI_filtered.txt", sep="\t
```

Before running it, open each file and check that the column names match what I used: `protein1`, `protein2`, `combined_score` in the links file, and `#string_protein_id`, `preferred_name` in the info file. If any differ, paste the first 3 lines of each file here and I'll adjust the code.

The `Weight` line is still the open question for your lead. Until he answers, you can't run the final analysis, but you can download these two files now.

STRING_PPI_filtered.txt

TXT

I got this after running the conversion script that u gave to me
what is ellipsis in weight column?
also, we need to make sure na the genes list that we have for both disease and drug targets it should be there in this file na like i mean we applied filters na like 0.700 etc. so we need to write a check script na or am thinking wrong?

Ran 2 commands, created a file, shared files ›

What "Ellipsis" is: it's my mistake. In my script I wrote `links["Weight"] = ...` as a placeholder meaning "fill this in later". In Python, `...` is a real built-in value called `Ellipsis`, so it didn't error and every row got the text "Ellipsis". Your file has 282,864 edges, all with that text. Don't run the precompute step on it, because `float("Ellipsis")` will crash. The `Weight` column needs real numbers, and that's still the question for your lead (how the author turned STRING scores into weights).

Your thinking is right. The filters can remove genes, so checking coverage is exactly the right step. I wrote `check_genes_in_string.py` and ran it on your files; it's saved as `check_genes_in_string.py`. It checks every disease gene and every drug target against the network and saves a report to `gene_coverage_check.csv`.

Results:

- The network has 17,525 genes and 282,864 edges.
- **Drug targets: all present.** BRAF (118 neighbours), MAP2K1 (81), MAP2K2 (41), EGFR (475).
- **Melanoma:** all 485 genes are in the network.
- **CRC:** 551 of 559 are in. Missing: VEGFA, C11orf65, TMEM238L, POU2AF3, POU2AF2, TTC22, METRNL, HOXA13.

A few missing disease genes is normal. The original example log also showed some dropped per disease, and the script just ignores those and warns. Possible reasons are no interactions at the 0.700 cutoff, or an old gene symbol (C11orf65, for example, may have been renamed).

VEGFA is the one I'd double-check. It's a well-studied gene and I'd expect it to have high-confidence partners. Run a quick search of `9606.protein.info.v12.5.txt.gz` for VEGFA. If it's listed under a different name or has no strong links at 0.700, that explains it. If it is listed and has plenty of links, then something went wrong in the mapping step. Tell me what you find, or send me the info file's first lines, and I'll help track it down.

Next steps: 1) get the Weight answer from your lead, 2) regenerate the STRING file with real weights, 3) run the precompute step. After that I can build `Drug_target.xlsx` and run both pairs.

since,
Network: 17525 genes, 282864 edges

[Melanoma] disease genes: 485 | in network: 485 | missing: 0
  drug target Dabrafenib -> BRAF: OK (degree 118)
  drug target Trametinib -> MAP2K1: OK (degree 81)
  drug target Trametinib -> MAP2K2: OK (degree 41)

[CRC] disease genes: 559 | in network: 551 | missing: 8
  missing: VEGFA, C11orf65, TMEM238L, POU2AF3, POU2AF2, TTC22, METRNL, HOXA13
  drug target Cetuximab -> EGFR: OK (degree 475)
  drug target Vemurafenib -> BRAF: OK (degree 118)

Full report saved to gene_coverage_check.csvs

Show more

so should i use medium confidence (0.400) or what? because we definitely want these in our network right?
also in that conversion script u gave me - remove ellipse in weight column - report what is reported by string like u said combined score na ------then we dont need conversion script right? i will ask the lead how did he converted it into weights? or is there any some standard formula or something or is it available on internet somewhere?

Comparing edge-weight conversions for the network script ›

Don't lower the cutoff just to get those 8 genes back. The cutoff should come from the method (what your lead used, or what's standard), not from whether a few genes happen to be present. 551 of 559 CRC genes is good coverage, and the original example run dropped 15 to 67 genes per disease and was fine with it. All your drug targets and all of Melanoma are already in the network.

Your lead's network also looks *stricter* than yours, not looser. His cache covered 12,268 genes, and yours at 0.700 has 17,525. So his file probably used a higher cutoff, or a different subnetwork or version. Moving to 0.400 would take you further from his setup and make the network even bigger. If you're curious, you could run 0.400 later as a sensitivity check, but not as your main result.

VEGFA is the odd one out. I'd still expect it to have high-confidence partners, so check it in the `protein.info` file before assuming it's a cutoff issue.

You still need the conversion script. It does two other jobs: it maps STRING protein IDs to gene symbols and joins them to the links. Only the `Weight` line changes.

If you just put the raw `combined_score` in `Weight`, the distances come out backwards. The precompute script uses `Weight` as a *distance* (shortest path = smallest total weight). With raw scores, a strong interaction (900) would count as *farther* than a weak one (700). Converting to a distance means flipping it. Common options (conventions, not a single official standard):

- `Weight = 1 - combined_score/1000`
- `Weight = -log(combined_score/1000)`
- no weights at all, which treats all edges equally

For what it's worth, the classic network-proximity method in the literature uses plain unweighted shortest paths. Your lead's script doesn't necessarily follow that, so I can't say which choice he made. I don't know of a single reference that fixes it either.

Ask your lead for two things, preferably his actual `STRING_PPI_filtered.txt`:

1. His actual file (or the exact cutoff, STRING version, network type, and weight formula). His cache and example results were built from it, and using it avoids guessing.
2. If he can't share it, which formula he used for `Weight`.

If he sends his file, we skip rebuilding it, and I can run the gene check on it directly. Until you hear back, I can prepare `Drug_target.xlsx` so you're ready to run.

9606.protein.info.v12.5.txt.gz
GZ

9606.protein.links.v12.5.min700.onlyAB.tsv.gz
GZ

can u give me updated conversion script & weight script also
and suppose if i have all the files then 100% correct script for doing user story like I can simply import already available script in Network_proximity folder right? or needs necessary changes any?
how to handle the missing genes like we cant ignore them na bec they are important so pls tell me that also

Ran 4 commands, created a file, shared files ›

I ran the conversion on your two STRING files and it works. `make_string_ppi.py` is the updated script, and the weight logic is built into it, so there's no separate weight script.

The conversion and weight script

The output has 282,864 edges and 17,525 genes, with a numeric `Weight` column and no blanks. The `WEIGHT_MODE` setting at the top has three options:

- `"hops"` **(my default):** every edge gets 1.0, so distance means the number of interaction steps. This is the classic unweighted network-proximity setup.
- `"score"`: `1 - combined_score/1000`, so stronger interactions are shorter.
- `"log"`: `-ln(combined_score/1000)`.

I can't tell which one your lead used, so treat the default as my assumption and confirm it with him. With a 0.700 minimum, the scores only range from 0.7 to 0.999, so the choice should matter less than it would on the full network.

Can you just import the existing Network_proximity scripts?

Mostly yes, but not with zero changes. I can't promise it's 100% correct, because I haven't run the full analysis yet and the biggest uncertainties are your lead's settings. Here's what needs attention:

1. **Run it in one folder.** Put `STRING_PPI_filtered.txt`, `Drug_target.xlsx`, and the two scripts together. The cache file `shortest_paths_cache.pkl` is created there by `precompute_shortest_paths.py`. The file names are hard-coded.
2. **Build `Drug_target.xlsx`.** Use sheet `Module_genes` with columns `Melanoma` and `CRC`, and sheet `Drug_combination` with the pairs. Write `MAP2K1,MAP2K2` without spaces.
3. **The degree calculation is a real bug, and I've now confirmed it.** On a test graph, the script's "degree" came out as 1 for every node whenever weights are non-zero, versus a true degree of 6 for the hub. When weights are 0 it counts the whole connected component instead. So the "degree-matched" random sets aren't actually degree-matched. The fix is small: compute degrees from the network graph (`G`) that `main()` already loads. Raise this with your lead before changing his script, or run his version unchanged and note the limitation.
4. **Memory.** The largest connected component has 17,336 genes, so the distance cache holds about 300 million pairs. That may need tens of GB of RAM in Python. Ask your lead for his `shortest_paths_cache.pkl` if he can share it, or check your machine first.
5. **Output is only a CSV and a log.** You still need the Excel conversion, and the PowerPoint step you'll get later.
6. **No random seed**, so repeated runs vary slightly.

Missing genes in CRC

You're right that they shouldn't just be ignored. But a gene that isn't a node in the network has no distances at all, so the maths can't use it. What you can do is recover what's recoverable and document the rest.

- **POU2AF2 and POU2AF3 are naming issues.** They are new symbols. STRING still lists these genes under their old names, C11orf53 and COLCA2, and both are in your network. I believe those are the right pairs, but please confirm on genenames.org. Rename them in the CRC gene list and you recover both.

- **Five genes are in STRING but have zero interactions at 0.700:** METRNL, TTC22, C11orf65, TMEM238L, HOXA13. They sit in the network as isolated points, so no distance can be computed for them. The honest options are to report them as excluded, or to run a second analysis at 0.400 as a sensitivity check (not as your main result).
- **VEGFA is the odd one.** It isn't in the STRING info file under any name, although VEGFB, VEGFC, and VEGFD are. I can't explain that, and since VEGFA is an important CRC gene, ask your lead about it. You could also download the `aliases` file (18.9 MB) from the same STRING page, or search VEGFA on string-db.org, to see whether it appears under another identifier. If it's genuinely absent, state that in your report.

After the rename fix, 6 of 559 CRC genes remain unusable. Add an "excluded genes and reason" table to your Excel/PPT output, because a reviewer will want to see it. The `gene_coverage_check.csv` from my earlier script already gives you that list.

Please send your lead these questions: the weight formula, the cutoff, the STRING version, whether he handled VEGFA, and ideally his `.pkl`. Once I have those, I can build `Drug_target.xlsx` and prepare the run.

i dont understand one thing like why we were checking disease module genes in string ppi like its protein-protein right? am confused actually

also list down all the questions i need to ask project lead now
also u didnt give me a run script for doing user story

Planning a run script for the drug-target workflow ›

Let me check the spreadsheet guidance before I write the Excel part of the run script.

Ran 2 commands, read a file, and 2 more tools ›

Why check disease genes against a protein–protein network?

You're not wrong to be confused. The mismatch is about naming. Your lists are **gene** names, and STRING's network is made of **proteins**, but each STRING protein is labelled by the symbol of the gene that codes for it. BRAF the gene → BRAF the protein. So the "disease genes" really mean "the proteins these genes make", and the drug targets (BRAF, EGFR, MAP2K1…) are proteins too.

The method asks: *how close, on the protein network, are the drug's target proteins to the disease proteins?* A protein that isn't a node in the network (or has no connections) has no distances, so it can't take part. That's why every disease gene and drug target must be found in the network.

Run script

`run_user_story.py` goes in the `Network_proximity` folder next to the original scripts. It doesn't change them, and it does the following:

1. `inputs`: builds `Drug_target.xlsx` from your Melanoma/CRC files, the two drug pairs, and the POU2AF2/POU2AF3 renames.
2. `run`: also builds the distance cache if missing, then runs `drug_synergy.py`.
3. `excel`: turns the CSV into a formatted Excel file with three sheets: **Results** (drug names, z-scores, separation, classification, plain-English meaning), **Excluded_genes** (with reasons), and **Run_info** (settings and legend).
4. `all`: does all of it.

I tested the input building on your real files, and tested the Excel step on a throwaway dummy CSV (deleted afterwards). I have **not** run the actual proximity analysis, because it needs the distance cache (memory-heavy) and your lead's confirmation of the settings. Edit the `RUN_NOTES` block in the script once you have those answers.

Questions for your project lead

About the network file

1. Can you share your `STRING_PPI_filtered.txt` and `shortest_paths_cache.pkl`? That avoids rebuilding and memory problems.
2. If not: which STRING version, network type (full or physical), and confidence cutoff did you use? His cache has 12,268 genes versus 17,525 at 0.700, which suggests he used something stricter.
3. How was the `Weight` column calculated (all 1, 1 − score, −log, or something else)?

About the method

4. In `get_degree_binning`, "degree" is counted as the number of nodes at distance exactly 0.0. With non-zero weights that's always just the node itself, so randomisation isn't degree-matched. Is that intended, or should degree come from the network?
5. Should the drug–disease p-values (`p_da`, `p_db`) affect the classification? Currently both branches give the same label.
6. Is it intended that each target is expanded to itself plus all its direct neighbours? Should I keep that for these drugs?
7. Should I set a random seed so results are reproducible?

About the inputs

8. Confirm the targets: Dabrafenib = BRAF; Trametinib = MAP2K1 + MAP2K2; Cetuximab = EGFR; Vemurafenib = BRAF (or should drugs' targets come from a source such as DrugBank?).
9. Where did the Melanoma and CRC gene lists come from, and should they be used as-is?
10. VEGFA is missing from the STRING info file, and five other CRC genes have no interactions at 0.700. How should those be handled: report as excluded, or also run a 0.400 sensitivity check?
11. Should the POU2AF2→C11orf53 and POU2AF3→COLCA2 renames be accepted? (I believe they're correct but you should verify on genenames.org.)

About the deliverables

12. What exactly should the Excel contain (just the results table, or extra sheets)?
13. Please send the PowerPoint script/template, and say whether the PPT should include the excluded-genes table and methods.
14. Any machine to run the precompute step on, given the likely RAM needs?

If he answers 1–3, I can help you run it right away.